EXP NO: 12

Implementation of Echo client Server

21/9/24

AIM

To implement echo client server using TCP / UDP sockets

ALGORITHM

i) TCP:

Echo Server and client:

* start the TCP echo server and set the server port to the given port or use 5000 by default.
* create a Server Socket
* wait for the client connection and create input and output streams for communication
* Read a message sent by the client and display the received message and send it back to the client with 'echo' added
* If the received message is bye, close connection other wise continue receiving messages.

ii) UDP

Echo Server and client

* Start the UDP server and listen on Port 66000
* start the UDP client and create a datagram packet

* start the UDP ve Read and round it into datagram packet and send it to the events.

* The server recieves the packet and extracts the message - the server. Then added "Echo" to the packet and sends back the response

* The client recieves the echoed response If the client sends bye, stop communication

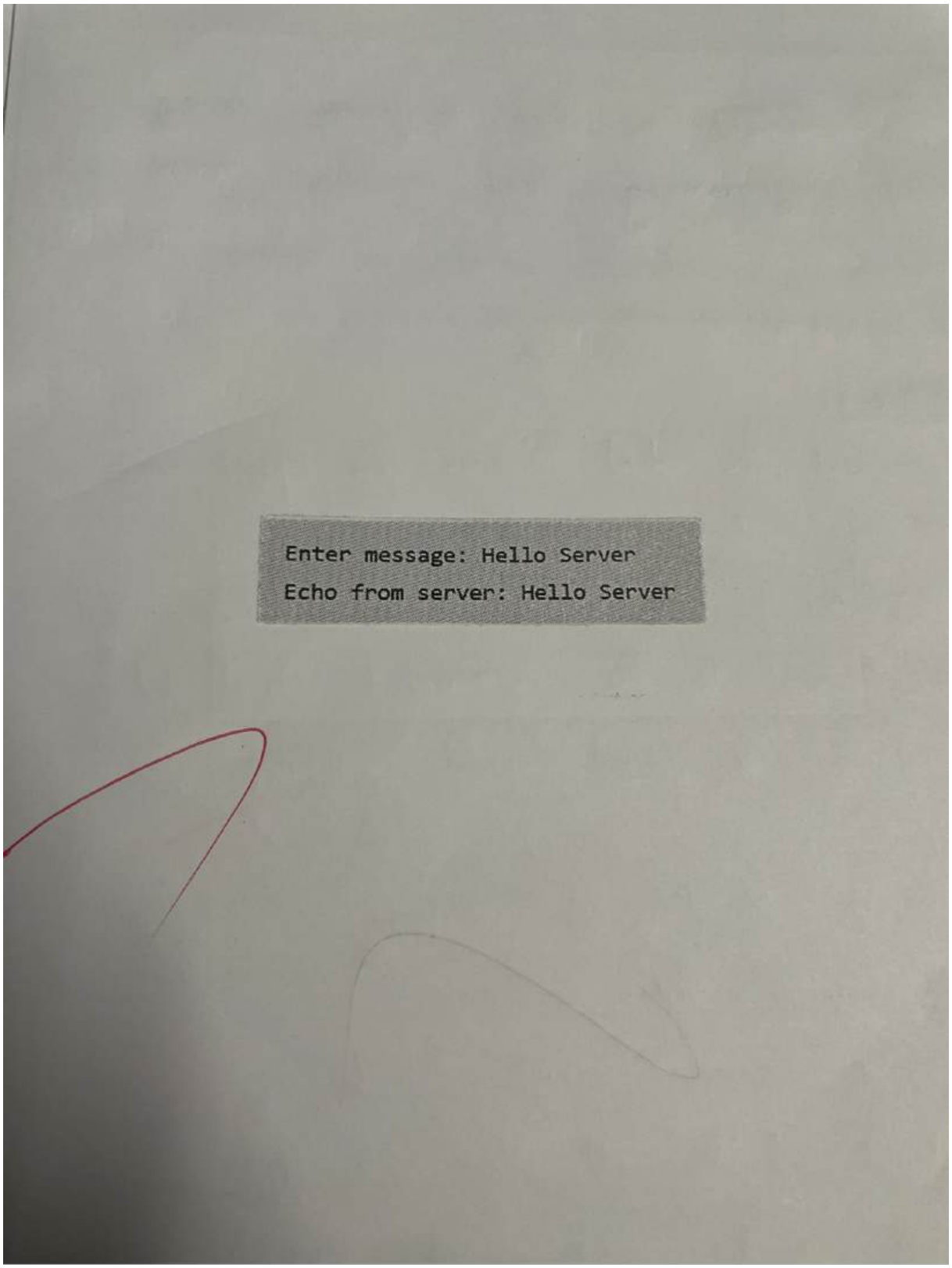
```
import socket

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.bind(('127.0.0.1', 12345))
s.listen(1)

print("TCP Server is waiting for client...")
conn, addr = s.accept()
print("Client connected:", addr)

data = conn.recv(1024).decode()
print("Client message:", data)

conn.send(data.encode())
conn.close()
s.close()
```


Enter message: Hello Server
Echo from server: Hello Server

```
import socket

client = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

msg = input("Enter message: ")
client.sendto(msg.encode(), ('localhost', 5001))

data, addr = client.recvfrom(1024)
print("Echo from server:", data.decode())

client.close()
```

Output:

```
UDP Server is ready...
Received: Hello Computer Networks
```

RESULT:

The Implement of Echo client server using TCP and UDP has been completed successfully.

EXP NO : 12 b)

Implementation of chat Program

2/9/24

AIM

To implement a chat program using socket programming that enables communication between a client and a server through a network.

PROCEDURE:

i) Start the chat server on port 5000 and wait for client connection

ii) Ask the client to enter a name and add client to the connected client list

iii) start the client and establish a socket

iv) the client sends message entered by the user to the server

v) The server recieves each message and broadcast it to all other connected clients

vi) client continuously listen for message from the server using a seperate listen thread

vii) If a client sends 'I quit', terminate that clients connection

vii) Remove the disconnected client and notify the remaining clients

viii) Continue accepting and managing new clients until the server is stopped.

Code -> CHAT SERVER

```
print("What does the Visualize button do?"import socket
s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.bind(('localhost', 5000))
s.listen(1)
print("TCP Chat Server Started...")
conn, addr = s.accept()
print("Client connected")
while True:
    msg = conn.recv(1024).decode()
    print("Client:", msg)
    if msg.lower() == "exit":
        break
    reply = input("Server: ")
    conn.send(reply.encode())
    if reply.lower() == "exit":
        break
conn.close()
s.close(
```

```
import socket
s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
s.bind(('localhost', 5001))
print("UDP Chat Server Started...")
while True:
    msg, addr = s.recvfrom(1024)
    msg = msg.decode()
    print("Client:", msg)
    if msg.lower() == "exit":
        break
    reply = input("Server: ")
    s.sendto(reply.encode(), addr)
    if reply.lower() == "exit":
        break
s.close()
```

OUTPUT

```
UDP Chat Server Started...
Client: Hello
Server: Hi
Client: How are you?
Server: I am fine
Client: exit
```

```
TCP Chat Server Started...
Client connected
Client: Hello
Server: Hi
Client: How are you?
Server: I am fine
Client: exit
```

RESULT:

The Implementation of chat - Program has been completed successfully.